Solutions Engineering Take-Home Assignment

Design an AI Agent for Customer Support

Time Estimate: ~4 hours

Submission Format: Slide deck + demo (e.g., recording, repo)

Overview

At Decagon, Solutions Engineers sit at the intersection of pre-sales and enterprise deployments — you build custom AI agents for customers *and* make them believers in what you built, translating complex technical decisions into clear value for everyone from the CX/product teams to the C-suite. For this exercise, you will design and prototype a conversational **chat or voice** AI agent for customer support. This assignment has two goals:

1. **Build it** - demonstrate familiarity with a language (e.g., Python, Typescript) of your choice to build an AI agent
2. **Sell it** - articulate the *why* behind your approach, explain the thesis of your agent architecture, and defend your key technical decisions

Some tips for the exercise:

- Code is expected, but doesn't need to be production-ready; usage of AI coding assistants are expected
- Avoid all-in-one agentic platforms that abstract away the implementation - we want to see you call APIs, design architecture, and orchestrate workflows directly.

Scenario

You're building a customer support agent for **Bookly**, a fictional online bookstore. Customers contact support for:

- Order status inquiries
- Return/refund requests
- General questions about shipping, policies, password reset, etc.

Requirements

Part 1: Prototype

Build a simple interactive demo. Users should be able to type or speak queries and receive responses. A web-based chat, CLI, or notebook all work fine.

Minimum requirements:

- At least one multi-turn interaction (e.g. collecting information before responding)
- At least one example of the agent taking an action or using a tool (real or mocked)
- At least one case where the agent chooses not to answer immediately and instead asks a clarifying question

You may use any LLM API (OpenAI, Anthropic, etc.) or mock responses, and any AI coding assistants.

Part 2: Solution Pitch Deck

Prepare a **3 - 5 slide deck** that walks through your agent architecture and defends your technical decisions. The goal is to articulate your thesis: why you build the Bookly agent the way you did, your key technical decisions, and why they were the right ones.

Your deck should cover:

- **The thesis** - what's your core belief about how a great CX agent should work, and how does your implementation reflect that?
- **Architecture** - how does a customer inquiry flow through your system? Cover the key components: agent orchestration, tools, memory, and prompts
- **Key decisions** - pick 2 - 3 choices that mattered most (e.g. how you handle tool use, hallucination prevention, guardrails, intent disambiguation). For each: what did you choose, what did you trade off, and why was it worth it?
- **What you'd do differently** - given more time or a production context, what's the first thing you'd change and why?

Deliverables

1. **Pitch Deck** (3 - 5 slides) - your thesis and defense of key technical decisions
2. **Prototype/Demo** - GitHub repo with run instructions, or a 2-minute screen recording

Tips

- **Prioritize depth over breadth.** A well-designed agent handling 2 use cases is better than a shallow one handling 10.
- **Have a point of view.** The best submissions make a clear argument, not just a description. We want to see how you think, not just what you built.
- **Keep the demo simple.** A basic UI that works is better than a fancy one that doesn't.
- **Mock what you need to.**

Questions?

If anything is unclear, make reasonable assumptions and document them. There’s no single “right” approach - we’re looking for clear thinking and reasonable tradeoffs.

Good luck! We're excited to see your approach.